SIGH — Sistema Integrado de Gestión de Horarios

Manual de Creación, Configuración y Despliegue en VirtualBox (Entorno Local)

Asignatura:	Bases de Datos 1 — Universidad El Bosque
Docente:	Ing. Christian Felipe Duarte
Responsable de Infraestructura:	David Alejandro Ordaz Luque
Componentes Desplegados:	PostgreSQL 18 (BD) + Spring Boot 4 (API) + Angular 21 (Web)
Entorno de Virtualización:	Oracle VM VirtualBox (2 Máquinas Virtuales en Adaptador Puente)

1. Arquitectura de Despliegue Local

Para simular fielmente el entorno que se desplegó en AWS EC2 pero dentro de la infraestructura local del estudiante, se crearon dos máquinas virtuales independientes sobre Oracle VM VirtualBox, enlazadas en la misma red local mediante un Adaptador Puente (Bridged Networking):

Parámetro	VM 1: Aplicación (Web + API)	VM 2: Base de Datos (RDBMS)
Nombre en VirtualBox	SIGH-APP-SERVER	SIGH-DB-SERVER
Sistema Operativo	Ubuntu Server 24.04 / 26.04 LTS (64-bit)	Ubuntu Server 24.04 / 26.04 LTS (64-bit)
Memoria RAM Asignada	2048 MB (2 GB)	1536 MB (1.5 GB)
Procesadores (vCPU)	2 Procesadores	1 Procesador
Disco Virtual (VDI)	20 GB (Dinámicamente asignado)	20 GB (Dinámicamente asignado)
Tipo de Adaptador de Red	Adaptador Puente (Bridged Adapter)	Adaptador Puente (Bridged Adapter)
IP Asignada en la Red	192.168.78.120	192.168.78.126
Componentes en Ejecución	Nginx (Puerto 80) + Spring Boot (8080)	PostgreSQL 18 (Puerto 5432)

2. Creación e Instalación de las Máquinas en VirtualBox

2.1. Creación de cada Máquina Virtual

1. Abrir Oracle VM VirtualBox y hacer clic en 'Nueva'.

2. Asignar nombre (ej. SIGH-DB-SERVER o SIGH-APP-SERVER), tipo Linux, versión Ubuntu (64-bit) y cargar la ISO oficial.

3. Marcar la casilla 'Omitir instalación desatendida' (Skip Unattended Installation) para control manual.

4. Asignar recursos: 1.5 a 2 GB de RAM y 1 a 2 CPUs.

5. Crear disco virtual VDI dinámico de 20 GB.

2.2. Configuración Esencial de Red (Adaptador Puente)

[IMPORTANTE: CONFIGURACIÓN DE RED] Por defecto VirtualBox configura la red en modo NAT, lo cual aísla la máquina virtual. Se debe cambiar obligatoriamente a 'Adaptador Puente' (Bridged Adapter) seleccionando la tarjeta Wi-Fi o Ethernet activa de tu computador. Esto permite que el anfitrión (Windows) y ambas máquinas virtuales compartan el mismo segmento de red.

2.3. Instalación de Ubuntu Server

1. Iniciar la máquina virtual y seguir el asistente de instalación estándar.
2. Configuración de red: Dejar DHCP automático; la máquina tomará una IP asignada por tu router (ej. 192.168.78.X).
3. Usuario y contraseña: Se estipuló el usuario 'user' con contraseña '1234'.
4. En la pantalla de SSH, marcar obligatoriamente la casilla '[X] Install OpenSSH server'.
5. Finalizar la instalación y reiniciar.

3. Preparación Inicial de las Máquinas Virtuales

3.1. En la VM de Base de Datos (192.168.78.126)

Ejecutar en la consola de la máquina para instalar PostgreSQL y habilitar SSH:

```
sudo apt update && sudo apt install -y openssh-server postgresql postgresql-contrib
sudo systemctl enable --now ssh
sudo ufw allow 22/tcp && sudo ufw allow 5432/tcp
hostname -I
```

3.2. En la VM de Aplicación (192.168.78.120)

Ejecutar en la consola de la máquina para instalar OpenJDK 17 y Nginx:

```
sudo apt update && sudo apt install -y openssh-server openjdk-17-jdk nginx curl
sudo systemctl enable --now ssh
sudo ufw allow 22/tcp && sudo ufw allow 80/tcp
hostname -I
```

4. Compilación de los Proyectos en Windows (Host)

Antes de transferir los artefactos, se compilan en la máquina de desarrollo Windows:

Compilación del Frontend (Angular 21):

```
cd C:\Users\dorda\IdeaProjects\proyecto_db\source\front-end\front-horarios
npm install
npm run build
# Genera: dist/front-horarios/browser/
```

Compilación del Backend (Spring Boot):

```
cd C:\Users\dorda\IdeaProjects\proyecto_db\source\back-end\api_horarios
.\gradlew.bat bootJar --no-daemon
# Genera: build/libs/apihorarios-0.0.1-SNAPSHOT.jar
```

5. Despliegue y Puesta en Marcha

Opción A: Despliegue Automatizado con deploy_vbox.ps1

Se desarrolló un script de orquestación que compila, transfiere y aprovisiona ambas máquinas automáticamente sin intervención manual. Desde la raíz del proyecto en PowerShell se ejecuta:

```
powershell -ExecutionPolicy Bypass -File .\deploy_vbox.ps1 -SkipBuild
```

Opción B: Despliegue Técnico Paso a Paso

Paso 1: 1. Configuración de PostgreSQL en la VM BD:

En /etc/postgresql/18/main/postgresql.conf se establece 'listen_addresses = *'.
En /etc/postgresql/18/main/pg_hba.conf se autoriza la red 'host all all 192.168.78.0/24 scram-sha-256'. Se reinicia con 'sudo systemctl restart postgresql'.

Paso 2: 2. Ejecución ordenada de scripts de BD:

Se suben los scripts de source/db/ y se ejecutan en orden: 01_ddl.sql, 02_catalogos.sql, 03_funciones_triggers.sql, 04_vistas.sql, 05_roles_permisos.sql (con app_password='1234'), 06_datos_prueba.sql y 07_pruebas_reglas.sql. El script 07 confirma que las 46 pruebas de reglas de negocio pasaron satisfactoriamente (46 | 46 | 0).

Paso 3: 3. Publicación del Frontend en Nginx:

Los archivos compilados de Angular se copian a /var/www/html con permisos para www-data.
En /etc/nginx/sites-available/default se configura la directiva 'try_files $uri $uri/ /index.html;' para soportar rutas internas de Angular (SPA) y el bloque 'location /api/ { proxy_pass http://localhost:8080/api/; }' para actuar como proxy inverso hacia Spring Boot.

Paso 4: 4. Configuración del Servicio Systemd para la API:

El archivo .jar se ubica en /home/user/app/api/ y se registra el servicio /etc/systemd/system/sigh-api.service inyectando las variables DB_HOST=192.168.78.126, DB_PORT=5432, DB_NAME=sigh y DB_PASSWORD=1234. Se inicia con 'sudo systemctl enable --now sigh-api'.

6. Verificación y Pruebas del Sistema

Una vez finalizado el despliegue, se validan los siguientes puntos de acceso en el navegador de Windows:

Componente	URL de Prueba	Resultado Esperado
Frontend Web (SPA)	http://192.168.78.120/	Carga completa del home de SIGH sin errores en consola.
Login Directo	http://192.168.78.120/#/login	Acceso interactivo con credenciales de prueba admin / 123456.
Estado de Conexión	http://192.168.78.120/#/conexion	Indicador verde confirmando conexión entre Angular y Spring Boot.
API Echo (Proxy Nginx)	http://192.168.78.120/api/echo	

		JSON con estado 'UP' y fecha del servidor.
Base de Datos (PostgreSQL)	192.168.78.126:5432 (sigh)	20 tablas y 7 vistas activas validadas con 46/46 reglas aprobadas.

7. Lecciones Aprendidas y Solución de Problemas (Troubleshooting)

Caso 1: 1. Error de ExecutionPolicy en PowerShell:

Windows bloquea scripts .ps1 externos por seguridad. Se solventa invocando powershell con '-ExecutionPolicy Bypass'.

Caso 2: 2. Error de Permisos en /etc/sudoers ('owned by uid..., should be 0'):

Si sudo se bloquea por permisos en la VM, se reinicia en 'Recovery Mode' de GRUB, se monta en lectura/escritura (mount -o remount,rw /) y se restaura con 'chown root:root /etc/sudoers && chmod 440 /etc/sudoers'.

Caso 3: 3. Socket UNIX de PostgreSQL Caído ('socket failed: No such file or directory'):

En instalaciones limpias donde el cluster main no inicia, se soluciona recreándolo con 'sudo pg_dropcluster 18 main --stop' y 'sudo pg_createcluster -u postgres -g postgres -e UTF-8 18 main --start'.

Caso 4: 4. Rutas Internas de Angular arrojando 404 en Nginx:

Al ser una SPA, el servidor web no tiene archivos físicos por cada ruta de Angular. Se soluciona mediante 'try_files $uri $uri/ /index.html;' en la configuración de Nginx.